

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №4
«Використання класів та методу поліморфізму»

Виконала:

Студентка групи ФЕП-11

Матла Діана Ярославівна

Перевірив:

Асистент Левуш Павло Назарович

Львів 2026

Мета: Метою роботи є вивчення принципів об'єктно-орієнтованого програмування мовою C++, зокрема реалізації інкапсуляції, успадкування, поліморфізму, перевизначення та перевантаження методів шляхом створення ієрархії класів для моделювання системи ігрових персонажів.

Хід роботи

Файли та готовий код було відправлено на github за посиланням:

<https://github.com/Diana-Matla/OOP/tree/main/lab%204>

Опис концепції програми

У даній роботі реалізовано ієрархію класів для моделювання системи ігрових персонажів із використанням принципів об'єктно-орієнтованого програмування. Основою системи є базовий клас `Character`, який містить спільні характеристики для всіх персонажів: ім'я, рівень, кількість здоров'я та силу атаки. Також базовий клас містить методи для отримання пошкоджень, виводу інформації та виконання атаки.

Від базового класу успадковуються похідні класи `Warrior` та `Mage`, які реалізують спеціалізовану поведінку персонажів. Клас `Warrior` описує фізичного бійця, який атакує мечем, а клас `Mage` — магичного персонажа, що використовує заклинання.

Для демонстрації глибшої ієрархії класів від класу `Mage` додатково успадковуються класи `FireMage` та `IceMage`, які реалізують різні типи магичних атак. Кожен із класів перевизначає метод `performAttack()`, демонструючи поліморфізм та різну поведінку об'єктів одного базового типу.

```

C character.h > ...
1  #ifndef CHARACTER_H
2  #define CHARACTER_H
3
4  #include <string>
5  using namespace std;
6
7  class Character
8  {
9  protected:
10     string characterName;
11     int level;
12     int healthPoints;
13     int attackPower;
14
15 public:
16     Character(string n, int l, int h, int ap);
17     virtual void performAttack();
18     virtual void MakeDamage();
19     virtual void displayInfo();
20 };
21
22
23 #endif

```

Базовий клас, character.h

```

C Warrior.h > ...
1  #ifndef WARRIOR_H
2  #define WARRIOR_H
3
4  #include "character.h"
5  #include <string>
6  using namespace std;
7
8  class Warrior : public Character
9  {
10 protected:
11     int strength;
12     string weaponType;
13 public:
14     Warrior(string n, int l, int h, int ap, int str);
15     void performAttack() override;
16     void MakeDamage() override;
17     void displayInfo();
18 };
19
20 #endif

```

Похідний від нього, Warrior.h

```

C> Archer.cpp > ...
1  #include <iostream>
2  #include <string>
3  #include <cmath>
4  #include "character.h"
5  #include "Archer.h"
6  using namespace std;
7
8  Archer::Archer(string n, int l, int h, int ap, int dist)
9  |   : Character(n, l, h, ap)
10 | {
11 |     distance = dist;
12 | }
13
14 void Archer::performAttack() {
15 |     cout << "Archer attacks using arrows being "<<distance<<" meters away"<<endl;
16 | }
17
18 void Archer::MakeDamage(){
19 |     int power = 0;
20 |     if ((distance>0)&&(distance<=50)) power = 5;
21 |     else power = 3;
22 |     cout<<"Attack damage:"<<attackPower*power<<endl;
23 | }
24
25 void Archer::displayInfo(){
26 |     Character::displayInfo();
27 |     cout << "Distance: "<< distance<< endl;
28 | }

```

Похідний від нього, archer.h

```

C Mage.h > ...
1  #ifndef MAGE_H
2  #define MAGE_H
3
4  #include "character.h"
5  #include <string>
6  using namespace std;
7
8  class Mage : public Character
9  | {
10 | protected:
11 |     int mana;
12 | public:
13 |     Mage(string n, int l, int h, int ap, int m);
14 |     void performAttack() override;
15 |     void MakeDamage()override;
16 |     void displayInfo();
17 | };
18
19 #endif

```

Похідний від нього, mage.h

```

C FireMage.h > FireMage > MakeDamage()
1  #ifndef FIREMAGE_H
2  #define FIREMAGE_H
3
4  #include "Mage.h"
5  #include <string>
6  using namespace std;
7
8  class FireMage : public Mage
9  {
10 private:
11     string spellType;
12 public:
13     FireMage(string n, int l, int h, int ap, int m);
14     void performAttack() override;
15     void MakeDamage() override;
16     void displayInfo();
17 };
18
19 #endif

```

Похідний від mage, firemage.h

```

C IceMage.h > IceMage > MakeDamage()
1  #ifndef ICEMAGE_H
2  #define ICEMAGE_H
3
4  #include "Mage.h"
5  #include <string>
6  using namespace std;
7
8  class IceMage : public Mage
9  {
10 public:
11     IceMage(string n, int l, int h, int ap, int m);
12     void performAttack() override;
13     void MakeDamage() override;
14     void displayInfo();
15 };
16
17 #endif

```

Похідний від mage, icemage.h

```

character.cpp > ...
1  #include <iostream>
2  #include <string>
3  #include "character.h"
4  using namespace std;
5
6  Character::Character(string n, int l, int h, int ap){
7      characterName = n;
8      level = l;
9      healthPoints = h;
10     attackPower = ap;
11 }
12
13
14 void Character::displayInfo(){
15     cout<<"-----"<<endl;
16     cout<<"Name:"<<characterName<<endl;
17     cout<<"Level:"<<level<<endl;
18     cout<<"Health points:"<<healthPoints<<endl;
19     cout<<"Attack power:"<<attackPower<<endl;
20 }
21
22 void Character::performAttack(){
23     cout<<"Character attacks"<<endl;
24 }
25
26 void Character::MakeDamage(){
27     cout<<"Attack damage:"<<attackPower<<endl;
28 }
29

```

Character.cpp

```

Warrior.cpp > displayInfo()
1  #include <iostream>
2  #include <string>
3  #include "character.h"
4  #include "Warrior.h"
5  using namespace std;
6
7  Warrior::Warrior(string n, int l, int h, int ap, int str)
8      : Character(n, l, h, ap)
9  {
10     strength = str;
11 }
12
13 void Warrior::performAttack() {
14     cout << "Warrior attacks with sword" << endl;
15 }
16
17 void Warrior::MakeDamage(){
18     cout<<"Attack damage:"<<attackPower*(strength/10)<<endl;
19 }
20
21 void Warrior::displayInfo(){
22     Character::displayInfo();
23     cout << "Strength: " << strength << endl;
24 }

```

Warrior.cpp

```

Archer.cpp > ...
1  #include <iostream>
2  #include <string>
3  #include <cmath>
4  #include "character.h"
5  #include "Archer.h"
6  using namespace std;
7
8  Archer::Archer(string n, int l, int h, int ap, int dist)
9  |   : Character(n, l, h, ap)
10 {
11 |   distance = dist;
12 }
13
14 void Archer::performAttack() {
15 |   cout << "Archer attacks using arrows being "<<distance<<" meters away"<<endl;
16 }
17
18 void Archer::MakeDamage(){
19 |   int power = 0;
20 |   if ((distance>0)&&(distance<=50)) power = 5;
21 |   else power = 3;
22 |   cout<<"Attack damage:"<<attackPower*power<<endl;
23 }
24
25 void Archer::displayInfo(){
26 |   Character::displayInfo();
27 |   cout << "Distance: "<< distance<< endl;
28 }

```

Archer.cpp

```

Mage.cpp > displayInfo()
1  #include <iostream>
2  #include <string>
3  #include "character.h"
4  #include "Mage.h"
5  using namespace std;
6
7  Mage::Mage(string n, int l, int h, int ap, int m)
8  |   : Character(n, l, h, ap)
9  {
10 |   mana = m;
11 }
12
13 void Mage::performAttack() {
14 |   cout << "Mage casts a magic spell" << endl;
15 }
16
17 void Mage::MakeDamage(){
18 |   cout << "Magic damage: "<< attackPower + mana<< endl;
19 }
20
21 void Mage::displayInfo(){
22 |   Character::displayInfo();
23 |   cout << "Mana: "<< mana<< endl;
24 }

```

Mage.cpp

```

IceMage.cpp > displayInfo()
1  #include <iostream>
2  #include <string>
3  #include "character.h"
4  #include "IceMage.h"
5  using namespace std;
6
7  IceMage::IceMage(string n, int l, int h, int ap, int m)
8      : Mage(n, l, h, ap, m){}
9
10 void IceMage::performAttack() {
11     cout << "IceMage casts ICE spell" << endl;
12 }
13
14 void IceMage::MakeDamage(){
15     cout << "Ice damage: " << attackPower + ((mana/10)*5) << endl;
16 }
17
18 void IceMage::displayInfo(){
19     Mage::displayInfo();
20     cout << "Magic type: Ice" << endl;
21 }

```

Icemage.cpp

```

main.cpp > main()
1  #include <iostream>
2  #include "Character.h"
3  #include "Warrior.h"
4  #include "Archer.h"
5  #include "Mage.h"
6  #include "FireMage.h"
7  #include "IceMage.h"
8
9  using namespace std;
10
11 int main() {
12
13     Warrior warrior("Thor", 10, 250, 30, 50);
14
15     Archer archer("Legolas", 8, 180, 25, 40);
16
17     Mage mage("Merlin", 12, 150, 20, 80);
18
19     FireMage fireMage("Ignis", 15, 140, 35, 100);
20
21     IceMage iceMage("Frost", 14, 160, 28, 90);
22
23
24     cout << "-----CHARACTERS ----- \n\n";
25
26     warrior.displayInfo();
27     cout << endl;
28
29     archer.displayInfo();
30     cout << endl;
31
32     mage.displayInfo();
33     cout << endl;
34
35     fireMage.displayInfo();
36     cout << endl;

```

Main.cpp

Вивід з терміналу

```
-----
Name:Ignis
Level:15
Health points:140
Attack power:35
Mana: 100
Magic type: Fire

-----

Name:Frost
Level:14
Health points:160
Attack power:28
Mana: 90
Magic type: Ice

                        BATTLE
-----

---- Warrior attack ----
Warrior attacks with sword
Attack damage:150

---- Archer attack ----
Archer attacks using arrows being 40 meters away
Attack damage:125
```

```
                        BATTLE
-----

---- Warrior attack ----
Warrior attacks with sword
Attack damage:150

---- Archer attack ----
Archer attacks using arrows being 40 meters away
Attack damage:125

---- Mage attack ----
Mage casts a magic spell
Magic damage: 100

---- Fire Mage attack ----
FireMage casts FIRE spell
Fire damage: 105

---- Ice Mage attack ----
IceMage casts ICE spell
Ice damage: 73

                        POLYMORPHISM TEST
-----
```

```
POLYMORPHISM TEST
-----

Warrior attacks with sword
Attack damage:150
-----

Archer attacks using arrows being 40 meters away
Attack damage:125
-----

Mage casts a magic spell
Magic damage: 100
-----

FireMage casts FIRE spell
Fire damage: 105
-----

IceMage casts ICE spell
Ice damage: 73
-----

-----

BATTLE END
PS D:\labs\OOP\lab 4> |
```

Висновок:

У ході виконання лабораторної роботи було створено ієрархію класів для моделювання ігрових персонажів із використанням механізмів об'єктно-орієнтованого програмування. Було реалізовано базовий клас Character та похідні класи Warrior, Mage, FireMage і IceMage. У роботі використано успадкування, поліморфізм, перевизначення та перевантаження методів. Створена система дозволяє демонструвати різну поведінку об'єктів через спільний інтерфейс базового класу, що підтверджує ефективність використання принципів ООП при розробці програмних систем.